

Module 05 Lab - Data Preparation **Objective:** To learn and apply the most common data preparation techniques. Raw data is rarely ready for a machine learning model.

- ✓ This process, also called preprocessing, is one of the most critical steps in the entire ML workflow. **In this lab, you will write more of the code.** Read the explanations and then complete the tasks in the code cells.

Part 1: Setup and Initial Look We will continue using the Titanic dataset because it has the exact problems we need to solve: missing values and non-numeric data.

```

◆ Gemini
import pandas as pd
import numpy as np
# Load the dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
# Let's look at the missing values
print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum())

--- Missing Values Before Cleaning ---
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64

```

Part 2: Handling Missing Values (Imputation) **Concept:** Most machine learning models cannot handle missing values (**NaN**). We must deal with them. Dropping the rows is an option, but you lose data. A better way is **imputation**, which means filling in the missing values with a calculated guess. Common imputation strategies:

- * **Mean:** Fill with the average value. Good for normally distributed data.
- * **Median:** Fill with the middle value. Better for skewed data or data with outliers (like **Fare**).
- * **Mode:** Fill with the most frequent value. Used for categorical data.

Task 1: Impute the 'Age' Column The 'Age' column is missing many values. Since age can be skewed (e.g., by a few very old passengers), using the **median** is a robust choice. **Your Task:** Calculate the median of the 'Age' column and use the `.fillna()` method to replace the missing values.

```

import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Calculate the median age
median_age = df['Age'].median()

# Fill missing Age values with the median
df['Age'] = df['Age'].fillna(median_age)

```

```
# Verify missing values
print(df['Age'].isnull().sum())
```

```
0
```

Part 3: Encoding Categorical Features
Concept: Machine learning models are mathematical, so they need numbers, not text. We need to convert categorical columns (like 'Sex' and 'Embarked') into a numerical format. The most common method is **One-Hot Encoding**. One-Hot Encoding takes a column with **N** categories and turns it into **N** new columns, each with a **1** or **0**. For example, the 'Sex' column (**male**, **female**) becomes two new columns: **Sex_male** and **Sex_female**. Pandas has a convenient function called `pd.get_dummies()` that does this for us.

Task 2: One-Hot Encode Categorical Columns
Your Task: Use `pd.get_dummies()` to encode the 'Sex' and 'Embarked' columns. Make sure to drop the original columns after encoding.

```
# One-hot encode the 'Sex' and 'Embarked' columns
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=False)

# View the updated dataframe
df.head()
```

PassengerId	Survived	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	Cabin	Sex_female	Sex_male	Embarked_C	
0	1	0	3	Braund, Mr. Owen Harris	22.0	1	0	A/5 21171	7.2500	NaN	False	True	False
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	0	PC 17599	71.2833	C85	True	False	True

Next steps: [Generate code with df](#) [New interactive sheet](#)

Part 4: Feature Scaling
Concept: Many models are sensitive to the scale of the features. For example, **Age** (from 0-80) and **Fare** (from 0-512) are on very different scales. This can cause the model to incorrectly believe that **Fare** is a more important feature simply because its values are larger. **Feature Scaling** solves this by putting all features on a similar scale. A common method is **Standardization** (`StandardScaler` in scikit-learn), which rescales the data to have a mean of 0 and a standard deviation of 1. **Important:** You only scale your numerical features, not your target variable or your newly encoded categorical columns.

Task 3: Scale the 'Age' and 'Fare' Columns
Your Task: Use `StandardScaler` from `sklearn.preprocessing` to scale the 'Age' and 'Fare' columns.

```
from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler = StandardScaler()

# Scale only the numerical columns
df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])
```

```
# View the updated dataframe  
df[['Age', 'Fare']].head()
```

	Age	Fare
0	-0.565736	-0.502445
1	0.663861	0.786845
2	-0.258337	-0.488854
3	0.433312	0.420730
4	0.433312	-0.486337



Knowledge CheckInstructions: Answer the following questions in this markdown cell.

****Why is it often better to impute missing values with the median instead of the mean?***

Explain in your own words what One-Hot Encoding does and why it is necessary.

3. Would you need to apply Feature Scaling to a Decision Tree model? Why or why not? (Hint: Think about how a Decision Tree makes its splits).**[ENTER YOUR ANSWERS HERE]**

- 1. Why is it often better to impute missing values with the median instead of the mean?** The median is less affected by extreme values or outliers in the data. If a feature is skewed, a few very large or very small values can pull the mean away from what is typical, while the median better represents the central tendency of most observations.
- 2. Explain in your own words what One-Hot Encoding does and why it is necessary.** One-Hot Encoding converts categorical text values into numerical columns so machine learning models can understand them. Each category becomes its own column with a value of 1 or 0, indicating whether that category is present. This is necessary because most ML models cannot work directly with text data and need numerical input.
- 3. Would you need to apply Feature Scaling to a Decision Tree model? Why or why not?** No, feature scaling is not required for Decision Tree models. Decision Trees make decisions by splitting data based on thresholds and comparisons, not distances or magnitudes. Because of this, they are not affected by differences in feature scale.